

CasADi session outline

- **Software stacks**
- CasADi
- Rockit

Joris Gillis, MECO, Mechanical Engineering, KU Leuven

From research idea to implementation

Dynamic Tube MPC: Learning Tube Dynamics with Massively Parallel Simulation for Robust Safety in Practice

William D. Compton¹, Noel Csomay-Shanklin¹, Cole Johnson², Aaron D. Ames¹

Abstract—Safe navigation of cluttered environments is a critical challenge in robotics. It is typically approached by separating the planning and tracking problems, with planning executed on a reduced order model to generate reference trajectories, and control techniques used to track these trajectories on the full order dynamics. Inevitable tracking error necessitates robustification of the nominal plan to ensure safety; in many cases, this is accomplished via worst-case bounding, which ignores the fact that some trajectories of the planning model may be easier to track than others. In this work, we present a novel method leveraging massively parallel simulation to learn a dynamic tube representation, which characterizes tracking performance as a function of actions taken by the planning model. Planning model trajectories are then optimized such that the dynamic tube lies in the free space, allowing a balance between performance and safety to be traded off in real time. The resulting *Dynamic Tube MPC* is applied to the 3D hopping robot ARCHER, enabling agile and performant navigation of cluttered environments, and safe collision-free traversal of narrow corridors.

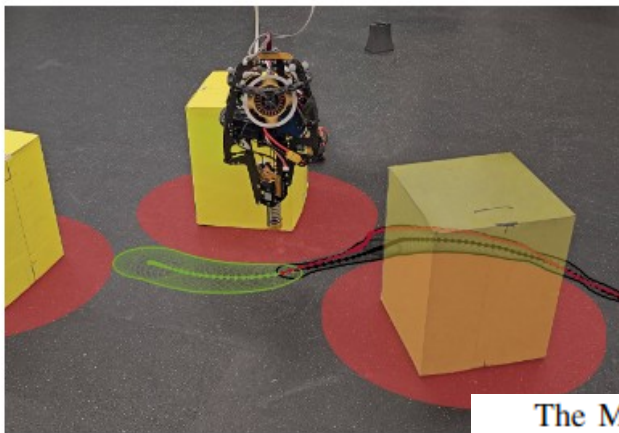


Fig. 1. ARCHER plans a collision-free path through clutter by jointly optimizing a path on a reduced order tube, whose dynamics are learned from simulation and tightly approximated by circles and buffered by the r

B. Dynamic Tube MPC

Given a dynamic tube model f_w^θ , which maps an error history and planning model trajectory to a tube prediction (either via one-shot or recursive application), we can formulate the Dynamic Tube MPC problem as a variant of (4):

$$\underset{\mathbf{v}(\cdot), \mathbf{z}(\cdot)}{\text{minimize}} \quad J(\mathbf{z}(\cdot), \mathbf{v}(\cdot)) \quad (9a)$$

subject to

$$\mathbf{z}_{j+1} = \mathbf{f}_z(\mathbf{z}_j, \mathbf{v}_j) \quad j \in [0, N-1], \quad (9b)$$

$$w_{0:N} = f_w^\theta(e_{-H:0}, \mathbf{z}_{-H:N}, \mathbf{v}_{-H:N}), \quad (9c)$$

$$\mathbf{z}_0 = \mathbf{z}_i, \quad (9d)$$

$$\mathbf{v}_j \in \mathcal{V} \quad j \in [0, N-1], \quad (9e)$$

$$B_{w_j}(\mathbf{z}_j) \in \mathcal{C} \quad j \in [0, N] \quad (9f)$$

The MPC problem is solved in CasADi [40] via SNOPT [41], with the neural network constraint implemented via L4CasADi [42], [43]. We successfully deploy our algorithm on the ARCHER hardware in a number of experimental trials. To achieve real time rates, we allow the first Dynamic Tube MPC solve to run to convergence, and then

From research idea to implementation

B. Dynamic Tube MPC

Given a dynamic tube model f_w^θ , which maps an error history and planning model trajectory to a tube prediction (either via one-shot or recursive application), we can formulate the Dynamic Tube MPC problem as a variant of (4):

$$\underset{\mathbf{v}(\cdot), \mathbf{z}(\cdot)}{\text{minimize}} \quad J(\mathbf{z}(\cdot), \mathbf{v}(\cdot)) \quad (9a)$$

subject to

$$\mathbf{z}_{j+1} = \mathbf{f}_z(\mathbf{z}_j, \mathbf{v}_j) \quad j \in [0, N-1], \quad (9b)$$

$$w_{0:N} = f_w^\theta(e_{-H:0}, \mathbf{z}_{-H:N}, \mathbf{v}_{-H:N}), \quad (9c)$$

$$\mathbf{z}_0 = \mathbf{z}_i, \quad (9d)$$

$$\mathbf{v}_j \in \mathcal{V} \quad j \in [0, N-1], \quad (9e)$$

$$B_{w_j}(\mathbf{z}_j) \in \mathcal{C} \quad j \in [0, N] \quad (9f)$$



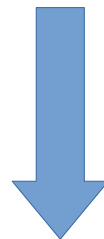
Computer code

From research idea to implementation

Specification of a *parametric* optimal control problem

Unknowns: states, controls, ...

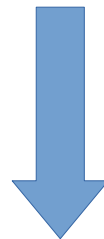
Direct transcription



Specification of a *parametric* nonlinear program

Decision variables: scalar, vector, matrix

Algorithmic differentiation



Callbacks for a numeric solver code

```
class IpoptUserClass : public TNLP {  
    bool eval_jac_g(Index n, const Number* x, bool new_x,  
                    Index m, Index nele_jac, Index* iRow, Index *jCol,  
                    Number* values) override;  
}
```

From research idea to implementation

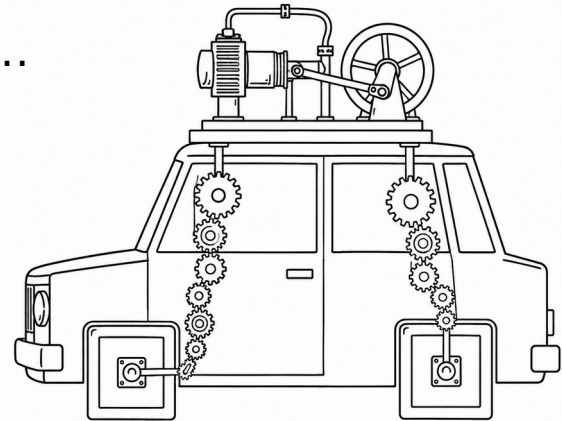
In 2026: just ask AI?

Problem:

In nonlinear programming, if you make an implementation mistake...
...you often still get an answer

Flawed solver,
Flawed Jacobian,
Flawed transcription,
...

Wrong answer,
Slow convergence,
Fragile convergence,
...



From research idea to implementation

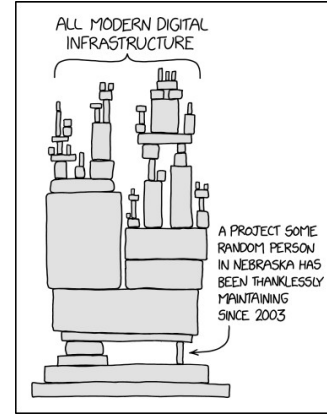
In 2026: just ask AI?

Sure, but make use of battle-tested infrastructure.
(This was equally true pre-AI)

“stack”

Abstraction

Simple user interface, simple semantics,
well tested.



xkcd

Pick your stack

Formal specification of a *parametric* optimal control problem



Formal specification of a *parametric* nonlinear program



Use a domain specific-language (e.g. AMPL, APMonitor...)

Use a toolbox in an existing programming language
(e.g. pyomo, JuMP, CasADi, ...)

Callbacks for a numeric solver code

https://github.com/meco-group/dynamic_optimization_inventory/blob/main/list.csv

Pick your stack

Nice to have's:

- Expressive enough to try crazy research ideas
- Not rigidly tied to any particular ecosystem
- Not too much performance penalty
- Free to use
- Auditable
- Alive/Stable across multiple generations of PhDs

Workshop stack

ImpaC++t

Impact

Formal specification of a *parametric* optimal control problem



Formal specification of a *parametric* nonlinear program



Callbacks for a numeric solver code

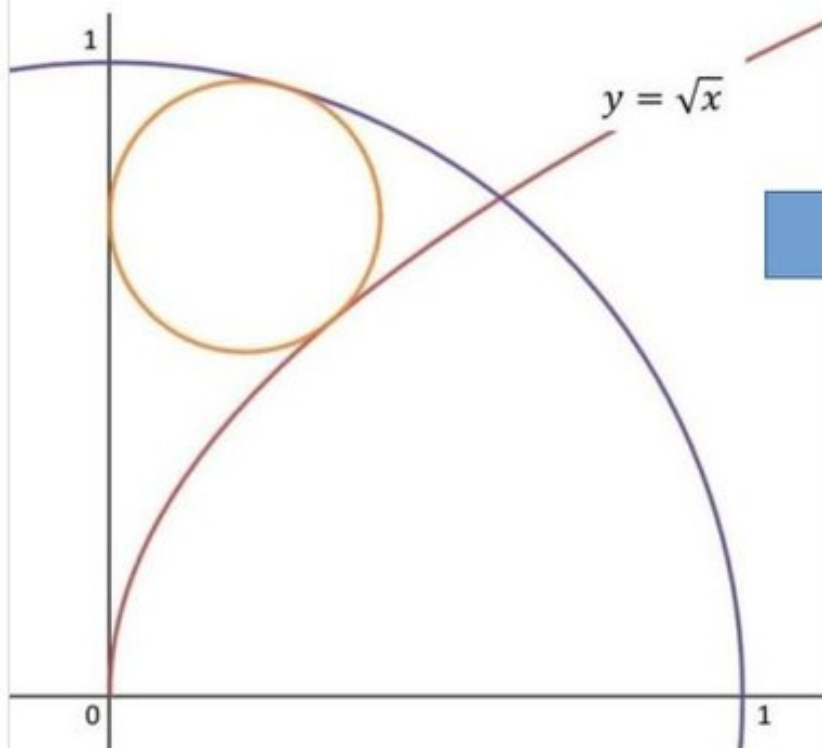




sir MIT, PhD in Vast Demented Atrophy
@iwontoffendyou

Hiring our first employee today. Gave this insanely hard problem for round 1, try it out if you're free:

What is the radius of smaller circle?



```
1 import casadi as ca
2 import numpy as np
3
4 opti = ca.Opti()
5
6 center = opti.variable(2)
7 radius = opti.variable()
8
9 opti.minimize(-radius)
10
11 # Sample edge vertices
12 ts = np.linspace(0, 2*np.pi, 1000)
13 v_x = radius*np.cos(ts)+center[0]
14 v_y = radius*np.sin(ts)+center[1]
15
16 opti.subject_to(v_x>=0)
17 opti.subject_to(v_y>=np.sqrt(v_x))
18 opti.subject_to(v_x**2+v_y**2<=1)
19
20 opti.solver("ipopt")
21 sol = opti.solve()
22
23 print(sol.value(radius)) # 0.21384
```

Goal of the session: convince you there is substance behind CasADi's front



CasADi session outline

- Software stacks
- **CasADi**
- Rockit

What is CasADi?



“Define system” → “Define problem based on system” → “Solve/deploy problem”

static/kinematic/dynamic

Initial value problem

Rootfinding problem

Nonlinear constrained optimization

Ipopt, Fatrop, Uno, SNOPT
Cvodes
OSQP, Highs
CPLEX, Gurobi
HPIPM...

CasADi stats

Automotive

Motorsports

Water management

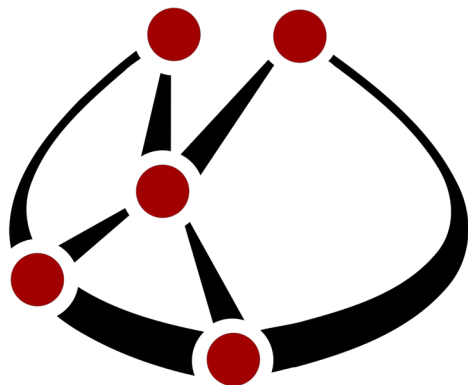
Wind energy

Pharmacology

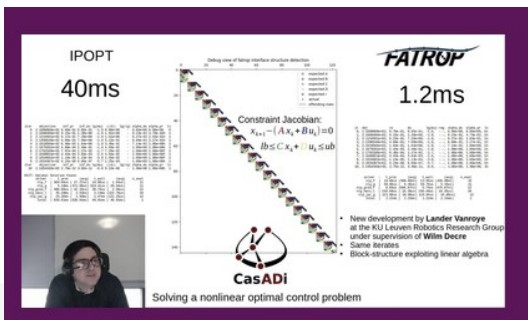
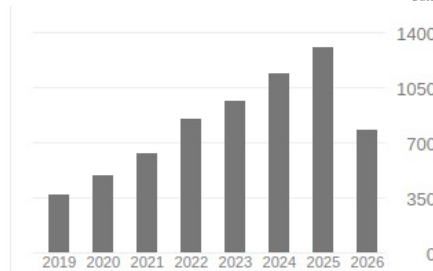
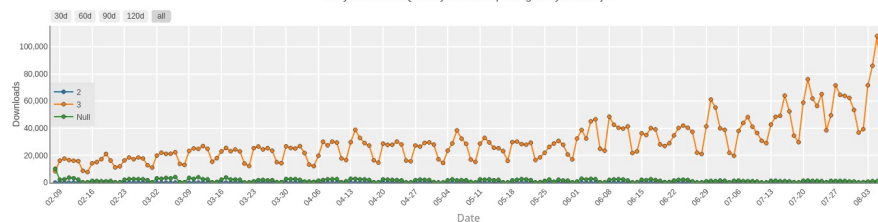
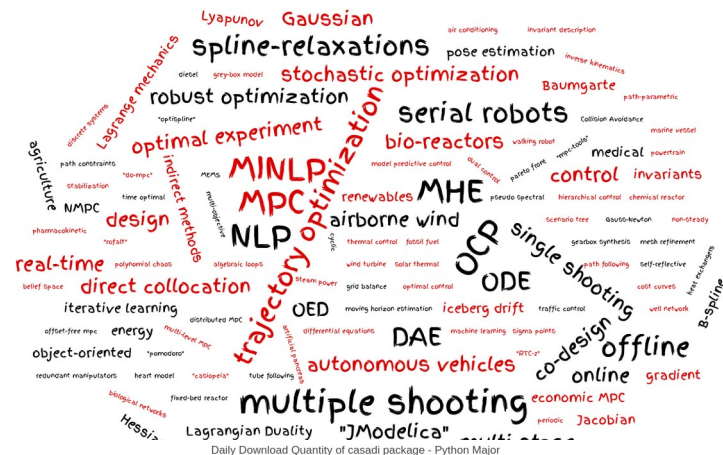
Robotics

HVAC

Agriculture



CasADi



TITLE	CITED BY	YEAR
CasADi: a software framework for nonlinear optimization and optimal control JAE Andersson, J Gillis, G Horn, JB Rawlings, M Diehl Mathematical Programming Computation, 1-36	5696	2018
A general-purpose software framework for dynamic optimization J Andersson Ph. D. thesis. Faculty of Engineering, KU Leuven, Leuven, Belgium	544	2013
CasADi: A Symbolic Package for Automatic Differentiation and Optimal Control J Andersson, J Åkesson, M Diehl Recent Advances in Algorithmic Differentiation, 297-307	466	2012
Handling uncertainty in economic nonlinear model predictive control: A comparative case	206	2014

Hands-on

```
pip install casadi
```

```
from casadi import *
```

```
x = SX.sym("x")
```

```
y = SX.sym("y")
```

```
print(jacobian(sin(x*y),x))
```

```
(cos((x*y))*y)
```

Hands-on

```
from casadi import *
```

```
x = SX.sym("x")
```

```
y = SX.sym("y")
```

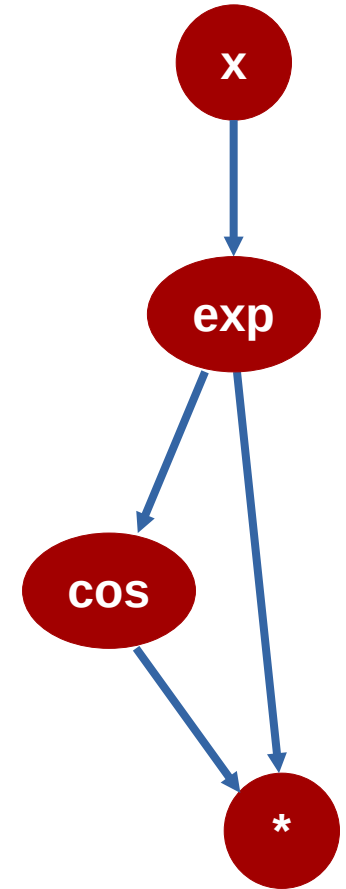
```
print(jacobian(sin(exp(x)),x))
```

```
@1=exp(x), (cos(@1)*@1)
```

CasADi recycles subexpressions

Graph ↔ Tree

classic computer-algebra has tree-like steps



Common subexpresion elimination

```
class ReactorModel:
    def heat_loss():
        return self.R*(self.T-self.T_amb)

    def excess_enthalpy(self):
        return self.m*self.Cp*(self.T-self.T_amb)
```

```
from casadi import *

x = SX.sym("x")
y = SX.sym("y")

print((x-y)+cos(x-y))
```

```
print(((x-y)+cos((x-y))))

print(cse((x-y)+cos(x-y)))

@1=(x-y), (@1+cos(@1))
```

Graphs can be optimized (more coming in CasADi 3.8)

How to represent vectors and matrices?

e.g. outer product of $a \in \mathbb{R}^3, b \in \mathbb{R}^2$

```
from casadi import *
```

```
a = SX.sym("a",3)
```

```
print(a.shape)
```

(3,1)

“Everything is a matrix”

How to represent vectors and matrices?

e.g. outer product of $a \in R^3, b \in R^2$

```
from casadi import *
```

```
a = SX.sym("a",3)
```

```
b = SX.sym("b",2)
```

```
R = a @ b.T
```

```
print(R.shape)
```

(3,2)

How to represent vectors and matrices?

e.g. outer product of $a \in \mathbb{R}^3, b \in \mathbb{R}^2$

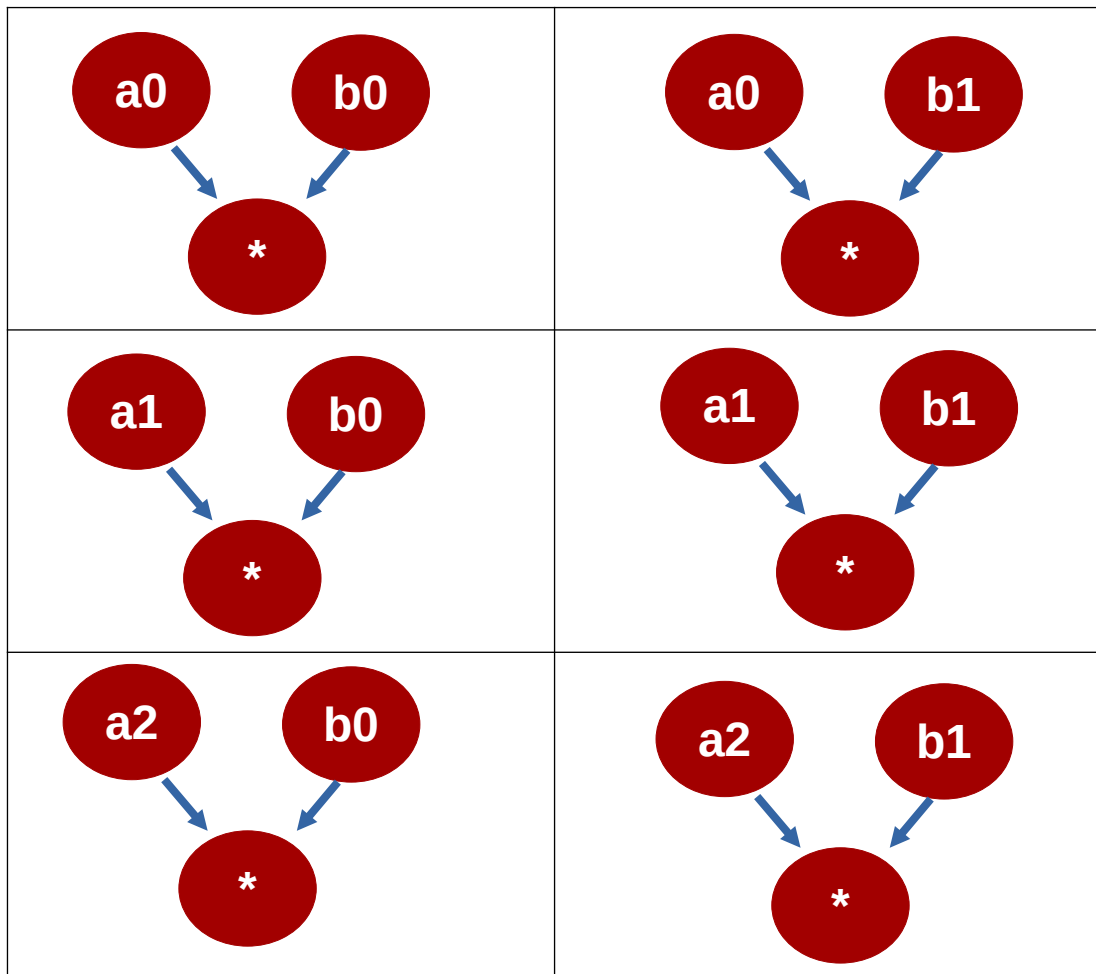
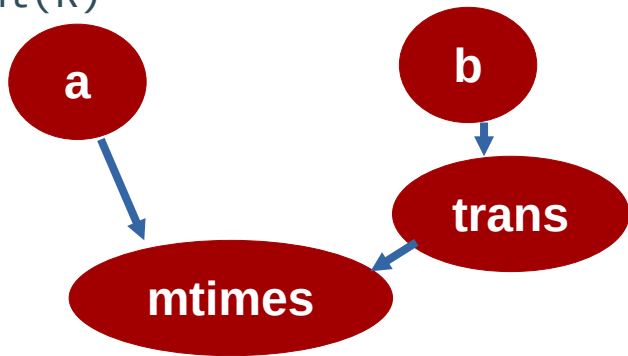
```
from casadi import *
```

```
a = SX.sym("a",3)
```

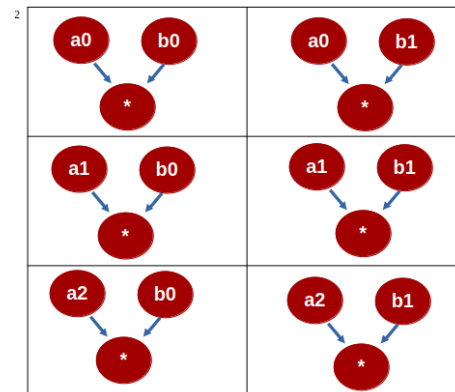
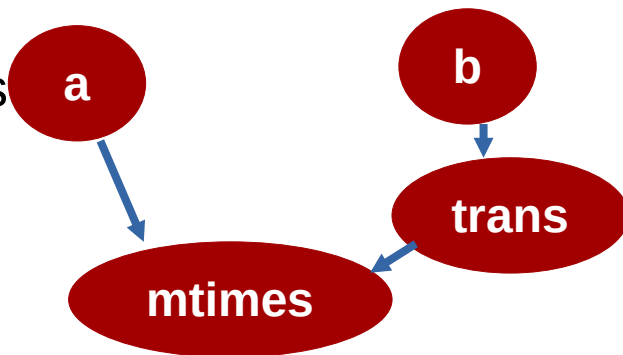
```
b = SX.sym("b",2)
```

```
R = a @ b.T
```

```
print(R)
```

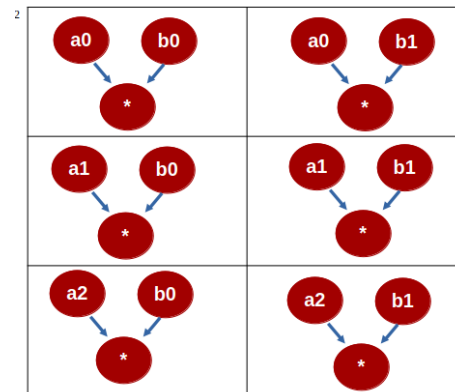
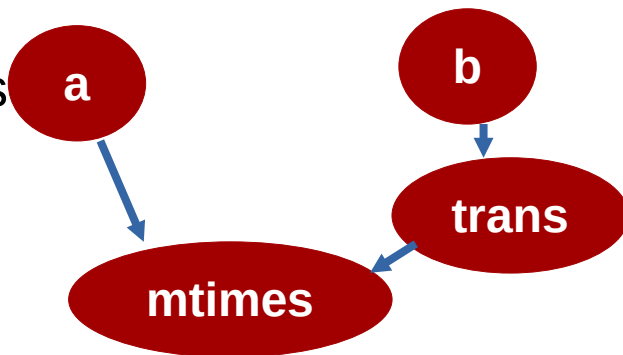


Matrix versus scalar graphs



Memory-efficient		
Overhead in evaluation		
Opportunities for simplification		
Ideal for	Partial differential equations	Lumped parameter systems
Typical in	PyTorch, Tensorflow, JAX	AMPL, Pyomo, JuMP

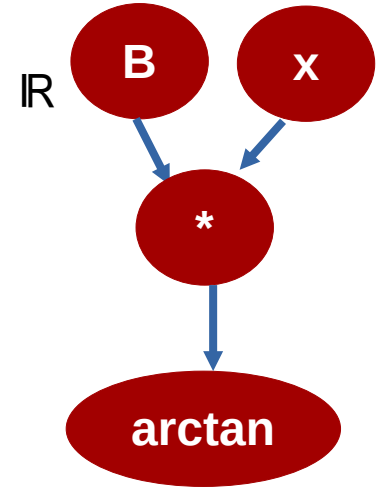
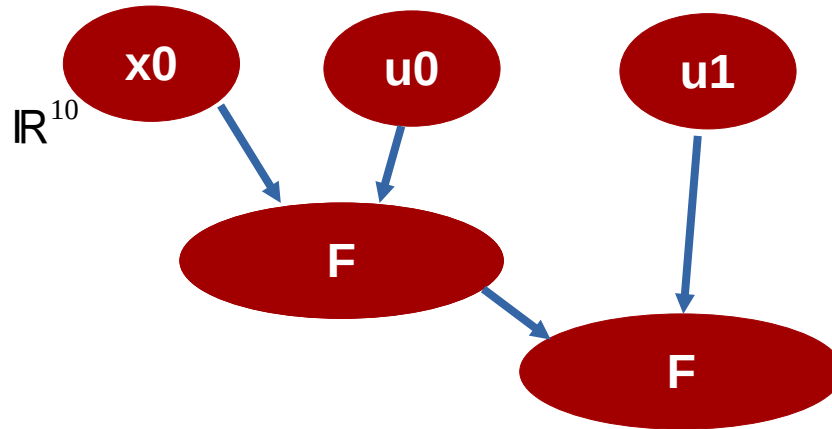
Matrix versus scalar graphs



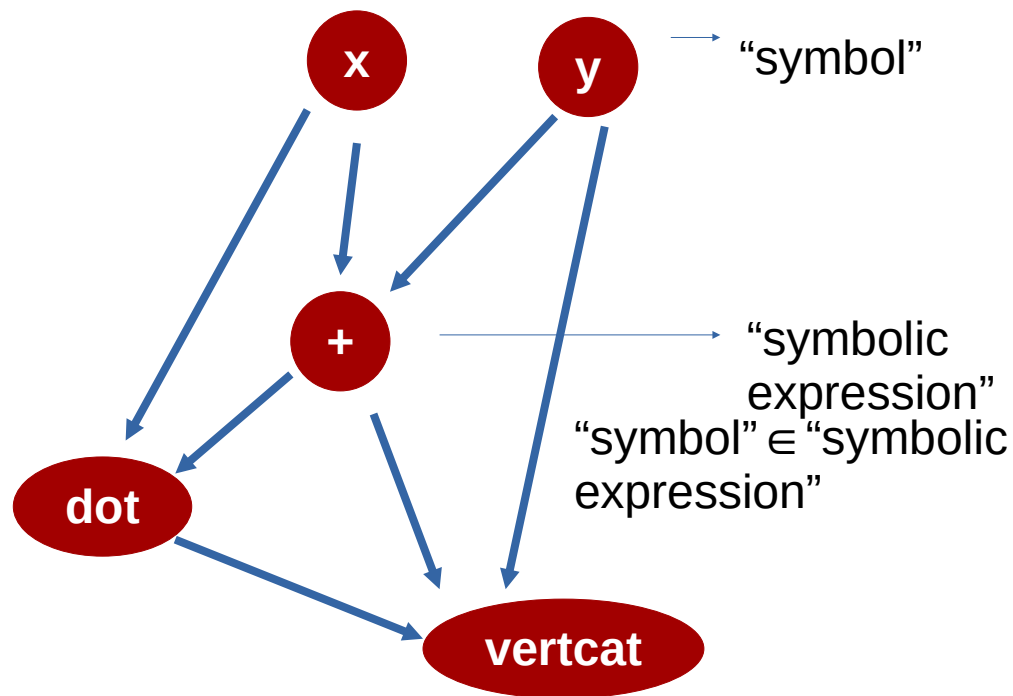
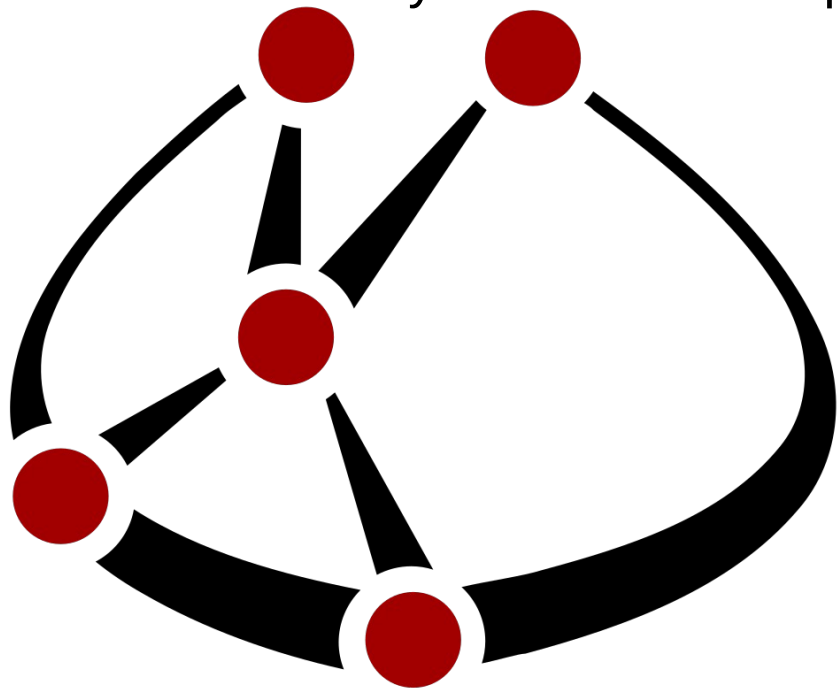
Memory-efficient		
Overhead in evaluation		
Opportunities for simplification		
Ideal for	Partial differential equations	Lumped parameter systems
Typical in	PyTorch, Tensorflow, JAX	AMPL, Pyomo, JuMP
CasADi has both	SX	MX

Best of both worlds

- Use SX for inner level modeling
 - e.g. right-hand side of a medium-scale heterogeneous system
 - Think “Pacejka tire model”
- Use MX for higher-level computation
 - e.g. initial value problem
 - Think “State of a race car after 2 samples”



You may have noticed expression graphs are important in CasADi



CasADi

Python objects referencing
a node in a directed acyclic graph (libcasadi.dll)

```
x = MX.sym('x', 2)
y = MX.sym('y', 2)
w1 = x+y
w2 = dot(x, w1)
w3 = vertcat(y, w1, w2)
```

Function as a read-only view on an expression graph

```
from casadi import *
```

```
x = MX.sym('x',2)
```

```
y = MX.sym('y',2)
```

```
w1 = x+y
```

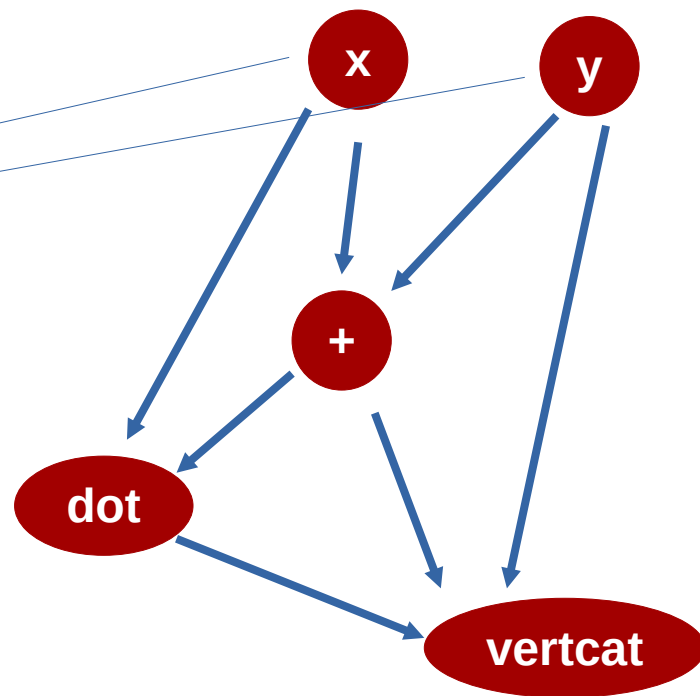
```
w2 = dot(x,w1)
```

```
w3 = vertcat(y,w1,w2)
```

```
F = Function('f',[x,y],[w2],[ 'x','y'],[ 'z'])
```

```
f:(x[2],y[2])->(z[5]) MXFunction
```

```
print(F([1,2],[3,4]))
```



Function as a read-only view on an expression graph

```
from casadi import *
```

```
x = MX.sym('x',2)
```

```
y = MX.sym('y',2)
```

```
w1 = x+y
```

```
w2 = dot(x,w1)
```

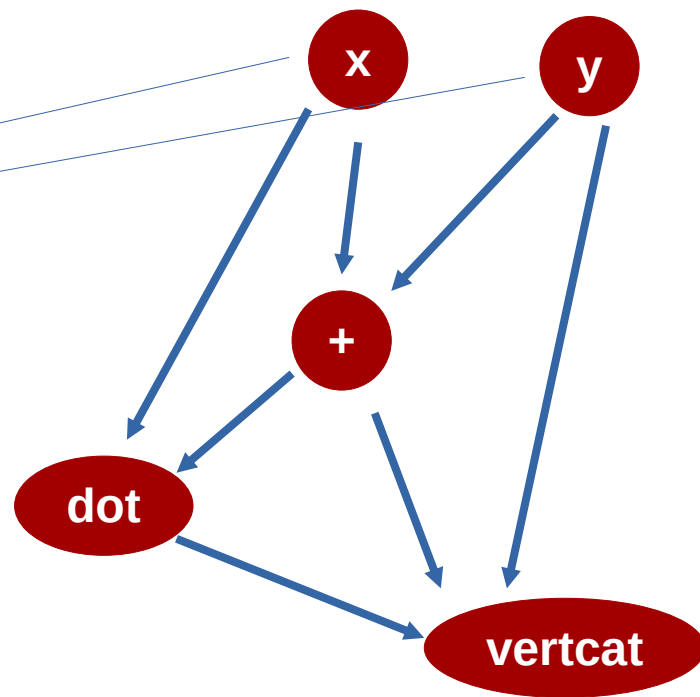
```
w3 = vertcat(y,w1,w2)
```

```
F = Function('f',[x,y],[w2],['x','y'],['z'])
```

```
f:(x[2],y[2])->(z[5]) MXFunction
```

```
print(F(x=[1,2],y=[3,4]))
```

```
{'z': DM(16)}
```



Function as a read-only view on an expression graph

```
from casadi import *
```

```
x = MX.sym('x',2)
y = MX.sym('y',2)
w1 = x+y
w2 = dot(x,w1)
w3 = vertcat(y,w1,w2)
```

```
F = Function('f',[x,y],[w2],['x','y'],['z'])
```

```
f:(x[2],y[2])->(z[5]) MXFunction
```

```
print(F([1,2],[3,4]))
```

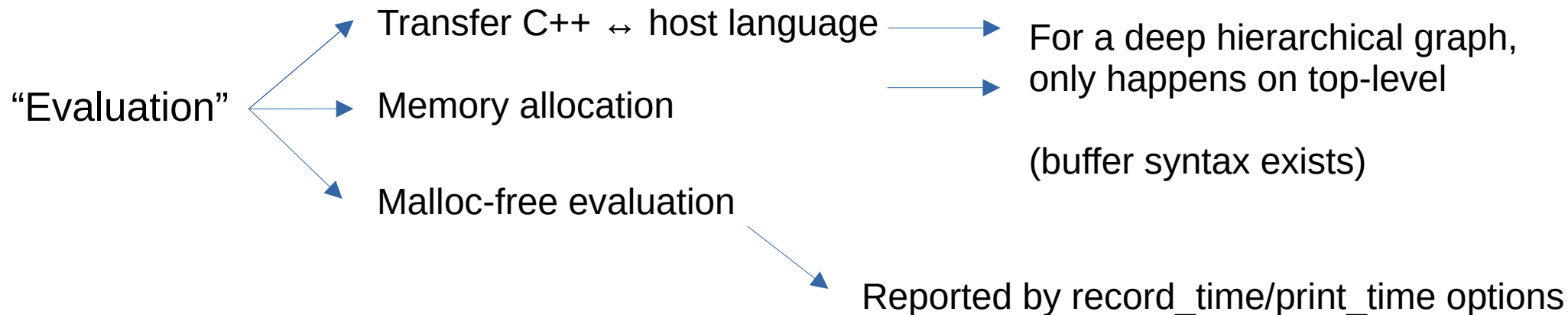
Design decisions by CasADi devs:

- *Construction* time not important
 - ‘scalable’ $O(\log(N)*N)$
- Optimize for *evaluation* time

“Construction”

“Evaluation”

Function details



```
F = Function('f', [x,y], [w2], ['x', 'y'], ['z'], {"print_time": True})
```

```
print(F([1,2], [3,4]))
```

f	:	t_proc	(avg)	t_wall	(avg)	n_eval
total		39.00us	(39.00us)	1.64us	(1.64us)	1
16						

Function details

```
F.disp(True)
```

```
f:(x[2],y[2])->(z) MXFunction
```

```
Algorithm:
```

```
@0 = input[0][0]
```

```
@1 = input[1][0]
```

```
@1 = (@0+@1)
```

```
@2 = dot(@0, @1)
```

```
output[0][0] = @2
```

```
F = Function('f',[x,y],[w2],['x','y'],['z'],  
{"print_instructions":True})
```

f:2: @1 = (@0+@1) inputs:

0: [1, 2]

1: [3, 4]

f:2: @1 = (@0+@1) outputs:

0: [4, 6]

f:3: @2 = dot(@0, @1) inputs:

0: [1, 2]

1: [4, 6]

f:3: @2 = dot(@0, @1) outputs:

0: 16

Direct Acyclic Graph gets sorted
depth-first-search into a virtual
machine

Function code-generation

MX

F.generate('f.c')

```
/* f:(x[2],y[2])->(z) */
```

```
int casadi_f0(const double** arg, double** res, int* iw, double* w, int mem) {
```

```
    int i;
```

```
    double *rr, *w0=w+0, *w1=w+2, w2;
```

```
    const double *cr, *cs;
```

```
    casadi_copy(arg[0], 2, w0);
```

```
    casadi_copy(arg[1], 2, w1);
```

```
    for (i=0, rr=w1, cr=w0, cs=w1; i<2; ++i)
        (*rr++) = ((*cr++)+(*cs++));
```

```
    w2 = casadi_dot(2, w0, w1);
```

```
    if (res[0]) res[0][0] = w2;
```

```
    return 0;
```

```
}
```

SX

F.expand().generate('f.c')

```
casadi_real a0, a1, a2;
```

```
a0=arg[0]? arg[0][0] : 0;
```

```
a1=arg[1]? arg[1][0] : 0;
```

```
a1=(a0+a1);
```

```
a0=(a0*a1);
```

```
a1=arg[0]? arg[0][1] : 0;
```

```
a2=arg[1]? arg[1][1] : 0;
```

```
a2=(a1+a2);
```

```
a1=(a1*a2);
```

```
a0=(a0+a1);
```

```
if (res[0]!=0) res[0][0]=a0;
```

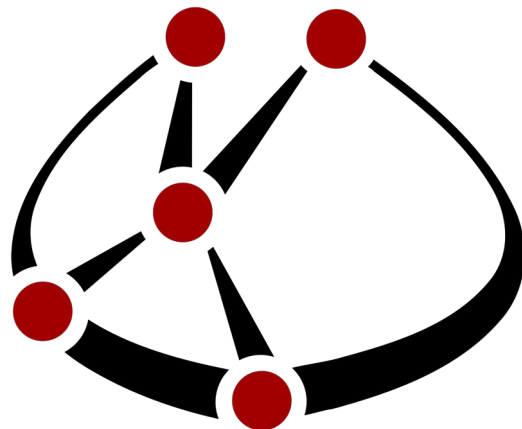
```
return 0;
```

Orientation



High-level view

Opti



What's in the middle?



Low-level view

Expression graphs, Function

CasADi

A

Exercise with Opti

casadi.org → Docs → Opti stack

<https://web.casadi.org/docs/#document-opti>

$$\min_{u_0, u_1, u_2, u_3} \left[\sum_{k=0}^3 u_k^2 + \sum_{k=0}^4 x_k^2 \right]$$

where $x_0 := 2$

$$x_{k+1} := x_k^2 + u_k, \quad k=0 \dots 3$$

subject to $x_k \geq 0, \quad k=1 \dots 3$

$$x_4 = 3$$

Expected objective value at optimum:
26.404

```
import casadi as ca
```

```
x = ca.MX.sym('x')
```

```
u = ca.MX.sym('u')
```

```
F = ca.Function('F', [x, u], [x**2+u])
```

```
opti = ca.Opti()
```

```
U = opti.variable(4)
```

```
x0 = 2
```

```
x1 = F(x0, U[0])
```

```
...
```

```
opti.subject_to(x4==3)
```

```
opti.minimize(ca.sumsqr(U)+...)
```

```
opti.solver('ipopt')
```

```
sol = opti.solve()
```

```
print(sol.value(opti.f))
```

```
U = opti.variable(4)
```

```
x0 = 2  
x1 = F(x0,U[0])  
x2 = F(x1,U[1])  
x3 = F(x2,U[2])  
x4 = F(x3,U[3])
```

```
opti.subject_to(x1>=0)  
opti.subject_to(x2>=0)  
opti.subject_to(x3>=0)  
opti.subject_to(x4==3)
```

```
X = ca.vertcat(x0,x1,x2,x3,x4)
```

```
opti.minimize(ca.sumsqr(U)  
+ca.sumsqr(X))
```

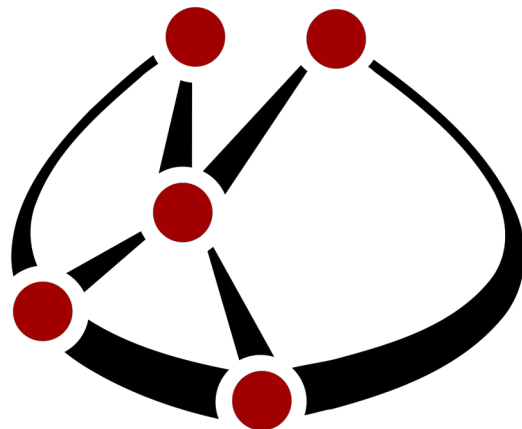
```
opti.solver('ipopt')  
sol = opti.solve()  
print(sol.value(opti.f))
```

Orientation



High-level view

Opti



What's in the middle?

Low-level view



Expression graphs, Function

CasADi

```
U = opti.variable(4)
```

```
x0 = 2  
x1 = F(x0,U[0])  
x2 = F(x1,U[1])  
x3 = F(x2,U[2])  
x4 = F(x3,U[3])
```

```
opti.subject_to(x1>=0)  
opti.subject_to(x2>=0)  
opti.subject_to(x3>=0)  
opti.subject_to(x4==3)
```

```
X = ca.vertcat(x0,x1,x2,x3,x4)
```

```
opti.minimize(ca.sumsqr(U)  
+ca.sumsqr(X))
```

```
opti.solver('ipopt')  
sol = opti.solve()  
print(sol.value(opti.f))
```

```
U = ca.MX.sym('U',4)
```

```
nlp = {'x': U}
```

```
x0 = 2  
x1 = F(x0,U[0])  
x2 = F(x1,U[1])  
x3 = F(x2,U[2])  
x4 = F(x3,U[3])
```

```
nlp['g'] = ca.vertcat(x1, x2, x3,x4)  
lbg = ca.vertcat(0, 0, 0, 3)  
ubg = ca.vertcat(inf,inf,inf,3)
```

```
X = ca.vertcat(x0,x1,x2,x3,x4)
```

```
nlp['f'] = ca.sumsqr(U)+ca.sumsqr(X)
```

```
solver = ca.nlpsol('solver','ipopt',nlp)  
print(solver(lbg=lbg,ubg=ubg))
```

What datatype is an nlp solver?

```
solver = ca.nlpсол('solver', 'ipopt', nlp)
print(type(solver))      # casadi.Function
print(solver)
```

```
solver:(x0[4], p[], lbx[4], ubx[4], lbг[4], ubг[4], lam_x0[4], lam_g0[4])
      ->(x[4], f, g[4], lam_x[4], lam_g[4], lam_p[]) IpoptInterface
```

```
print(solver(lbg=lbг, ubg=ubг))
```

```
{'f': DM(26.4039), 'g': DM([1.2962, 1.13715, 1.55434, 3]), 'lam_g':
DM([-1.049e-05, -1.5354e-05, -8.39538e-06, -7.16806]), 'lam_p':
DM(0x0), 'lam_x': DM([0, 0, 0, 0]), 'x': DM([-2.7038, -0.542986,
0.261233, 0.584031])}
```

Generic syntax

Dictionary with symbolic expressions

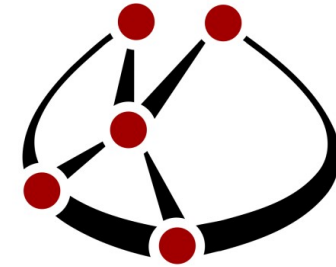
`solver(label,plugin_name,problem_struct,options_struct)`

rootfinder
integrator
qpsol
nlpsol

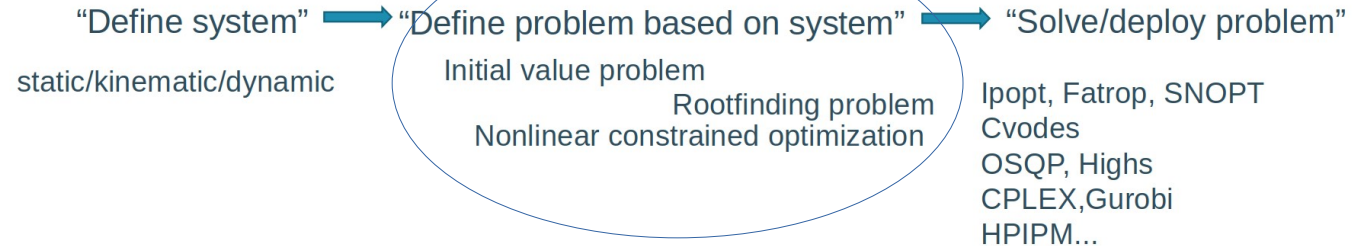
Dynamically searched for `casadi_<solver>_<plugin_name>.dll`
e.g. `casadi_nlpsol_casadi.dll`

```
solver(label,plugin_name,problem_struct,options_struct)
```

What is CasADi?



CasADi



Printed by
Ipopt

This is Ipopt version 3.14.19, running with linear solver MUMPS 5.8.2.

Number of nonzeros in equality constraint Jacobian...: 4
Number of nonzeros in inequality constraint Jacobian.: 6
Number of nonzeros in Lagrangian Hessian.....: 10

Total number of variables.....: 4
Total number of equality constraints.....: 1
Total number of inequality constraints.....: 3
inequality constraints with only lower bounds: 3

iter	objective	inf_pr	inf_du	lg(mu)	d	lg(rg)	alpha_du	alpha_pr	ls
0	4.2950331e+09	6.55e+04	1.21e-01	-1.0	0.00e+00	-	0.00e+00	0.00e+00	0
...									
76	2.6403948e+01	4.97e-06	4.63e-12	-12.9	1.96e-03	-	1.00e+00	1.00e+00h	1

Number of Iterations.....: 76
...

Printed by
CasADi

EXIT: Optimal Solution Found.

	solver	:	t_proc	(avg)	t_wall	(avg)	n_eval
	nlp_f		4.89ms	(57.51us)	322.59us	(3.80us)	85
	nlp_g		6.67ms	(78.47us)	404.13us	(4.75us)	85
	nlp_grad_f		8.56ms	(109.74us)	584.88us	(7.50us)	78
	nlp_hess_l		26.53ms	(349.14us)	1.84ms	(24.21us)	76
	nlp_jac_g		11.91ms	(152.74us)	836.55us	(10.72us)	78
	total		349.97ms	(349.97ms)	24.65ms	(24.65ms)	1

CasADi Function, generated by CasADi on request by Ipopt

EXIT: Optimal Solution Found.

solver	:	t_proc	(avg)	t_wall	(avg)	n_eval
nlp_f		4.89ms	(57.51us)	322.59us	(3.80us)	85
nlp_g		6.67ms	(78.47us)	404.13us	(4.75us)	85
nlp_grad_f		8.56ms	(109.74us)	584.88us	(7.50us)	78
nlp_hess_l		26.53ms	(349.14us)	1.84ms	(24.21us)	76
nlp_jac_g		11.91ms	(152.74us)	836.55us	(10.72us)	78
total		349.97ms	(349.97ms)	24.65ms	(24.65ms)	1

Printed by
CasADi

CasADi Function, generated by CasADi on request by IpoptInterface

EXIT: Optimal Solution Found.

solver	:	t_proc	(avg)	t_wall	(avg)	n_eval
nlp_f		4.89ms	(57.51us)	322.59us	(3.80us)	85
nlp_g		6.67ms	(78.47us)	404.13us	(4.75us)	85
nlp_grad_f		8.56ms	(109.74us)	584.88us	(7.50us)	78
nlp_hess_l		26.53ms	(349.14us)	1.84ms	(24.21us)	76
nlp_jac_g		11.91ms	(152.74us)	836.55us	(10.72us)	78
total		349.97ms	(349.97ms)	24.65ms	(24.65ms)	1

Printed by
CasADi

Any CasADi Function can be used in an MX graph

```
import casadi as ca
```

```
x = ca.MX.sym('x')
```

```
u = ca.MX.sym('u')
```

```
F = ca.Function('F', [x, u], [x**2+u])
```

```
opti = ca.Opti()
```

```
U = opti.variable(4)
```

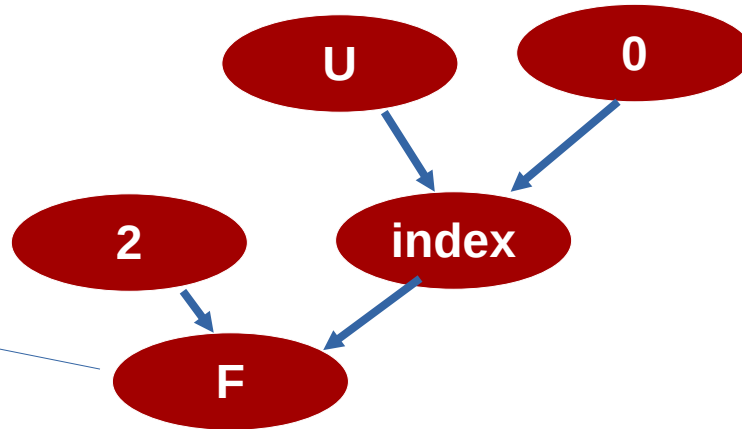
```
x0 = 2
```

```
x1 = F(x0, U[0])
```

```
...
```

```
opti.subject_to(x4==3)
```

```
....
```



Any CasADi Function can be used in an MX graph

```
import casadi as ca
```

```
x = ca.MX.sym('x')
```

```
u = ca.MX.sym('u')
```

```
F = ca.Function('f', [x, u], [1])
```

```
opti = ca.Opti()
```

```
U = opti.variable(4)
```

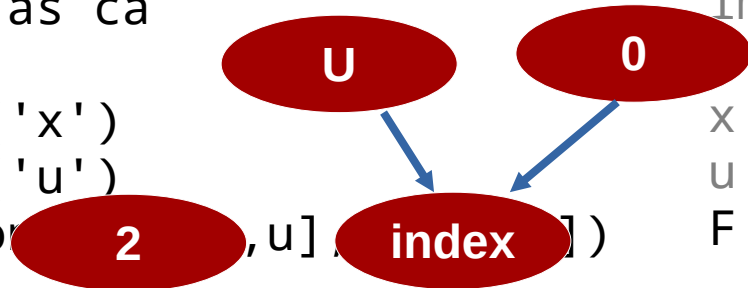
```
x0 = 2
```

```
x1 = F(x0, U[0])
```

```
...
```

```
opti.subject_to(x4==3)
```

```
....
```



```
import casadi as ca
```

```
x = ca.MX.sym('x')
```

```
u = ca.MX.sym('u')
```

```
F = integrator(...)
```

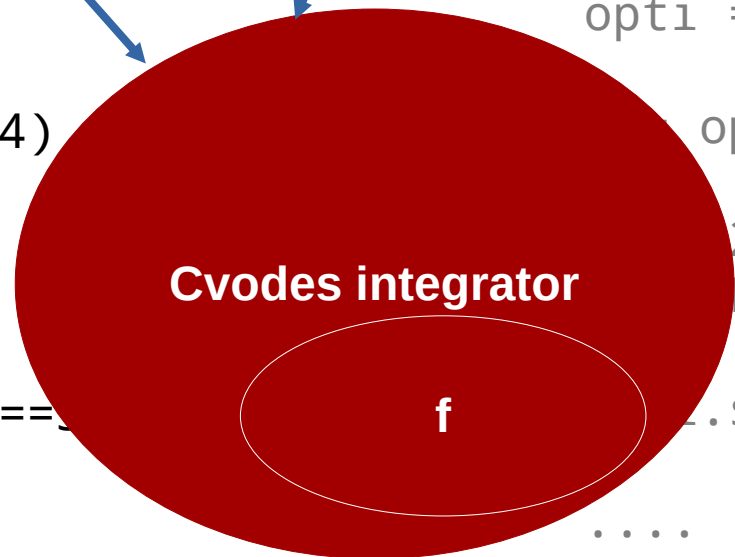
```
opti = ca.Opti()
```

```
opti.variable(4)
```

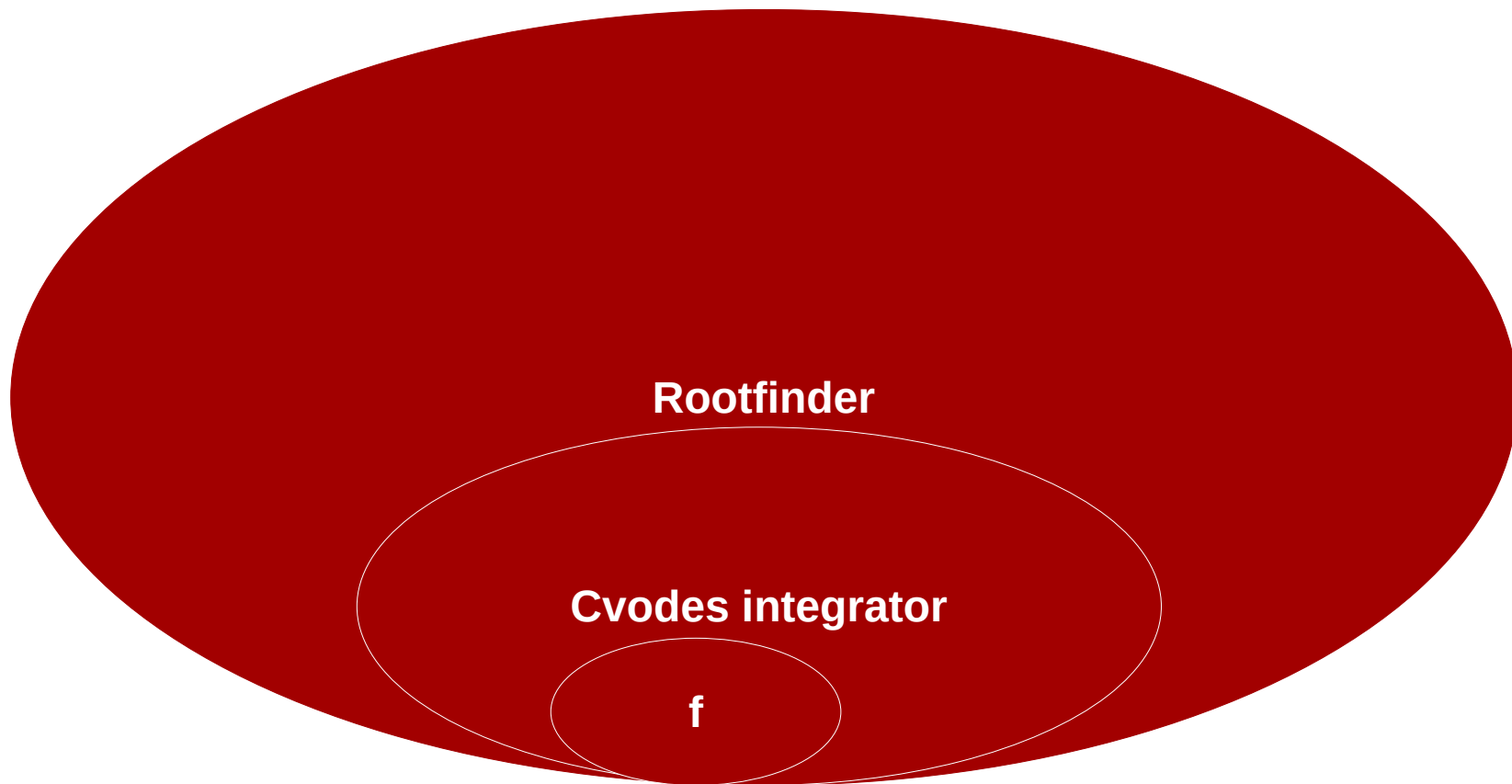
```
x0 = 2  
F(x0, U[0])
```

```
...subject_to(x4==3)
```

```
....
```



Any CasADi Function can be used in an MX graph

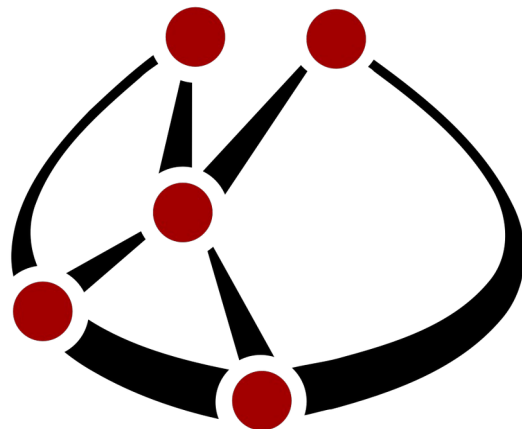


Orientation



High-level view

Opti



What's in the middle?

Low-level view



Expression graphs, Function

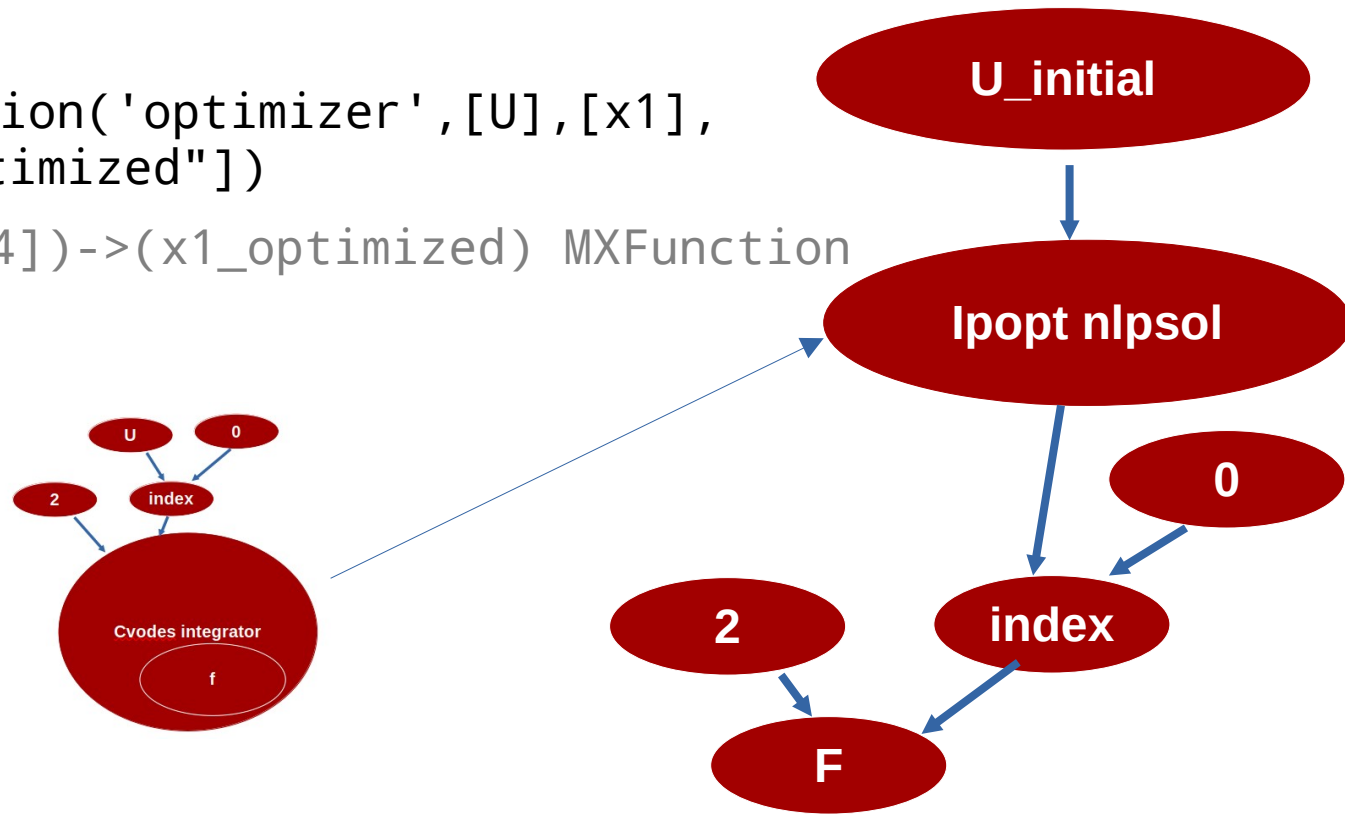
CasADi

```
U = opti.variable(4)
x0 = 2
x1 = F(x0,U[0])
```

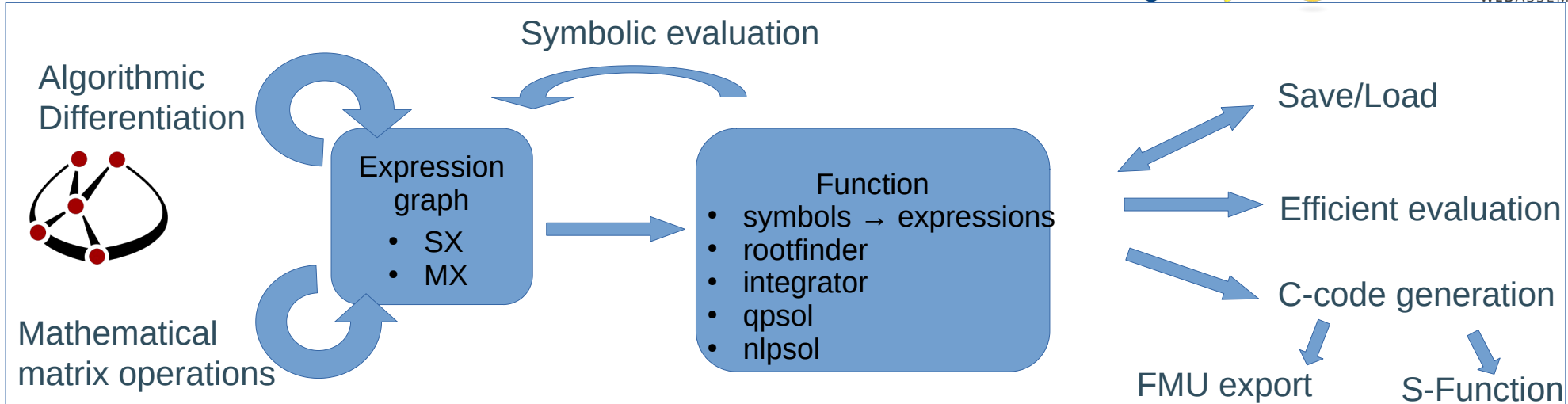
```
opti.minimize(ca.sumsqr(U)+ca.sumsqr(X))
```

```
opti.solver('ipopt')
```

```
optimizer = opti.to_function('optimizer',[U],[x1],
    ["U_initial"],["x1_optimized"])
    optimizer:(U_initial[4])->(x1_optimized) MXFunction
print(optimizer)
```



Overview



Alternative stacks



ImpaC++t

Impact

ACADOS

DO-MPC

MPC-TOOLS

OMG-TOOLS

FORCESPRO

OPTAS

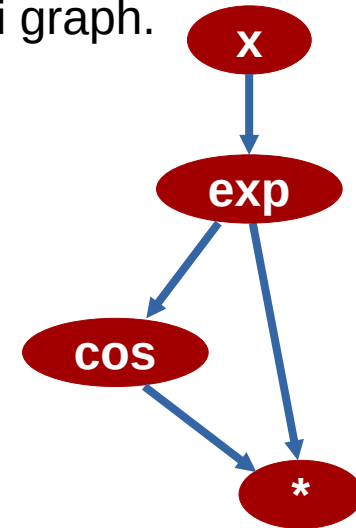


https://github.com/meco-group/dynamic_optimization_inventory/blob/main/list.csv

A recipe for slow code?



- 50KLOC class-full Matlab code?
- At the end of the day, your number crunching core is 'just' a CasADi graph.
 - Cost-free abstractions
 - CasADi ~ compiler





Nice to have's:

- **Expressive enough to try crazy research ideas**
- Not rigidly tied to any particular ecosystem
- Not too much performance penalty
- Free to use
- Auditable
- Alive/Stable across multiple generations of PhDs

- Do it yourself: maximum flexibility



Nice to have's:

- Expressive enough to try crazy research ideas
- **Not rigidly tied to any particular ecosystem**
- Not too much performance penalty
- Free to use
- Auditable
- Alive/Stable across multiple generations of PhDs

- Generated code artifacts
- Transfer Function/graphs between programming languages



Nice to have's:

- Expressive enough to try crazy research ideas
- Not rigidly tied to any particular ecosystem
- **Not too much performance penalty**
- Free to use
- Auditable
- Alive/Stable across multiple generations of PhDs

- Cost-free abstractions



Nice to have's:

- Expressive enough to try crazy research ideas
- Not rigidly tied to any particular ecosystem
- Not too much performance penalty
- **Free to use**
- Auditable
- Alive/Stable across multiple generations of PhDs

- Open-source LGPLv3
- Free to use commercially



Nice to have's:

- Expressive enough to try crazy research ideas
- Not rigidly tied to any particular ecosystem
- Not too much performance penalty
- Free to use
- **Auditable**
- Alive/Stable across multiple generations of PhDs

- Open-source
- Functions can be walked
- Generated code can be inspected

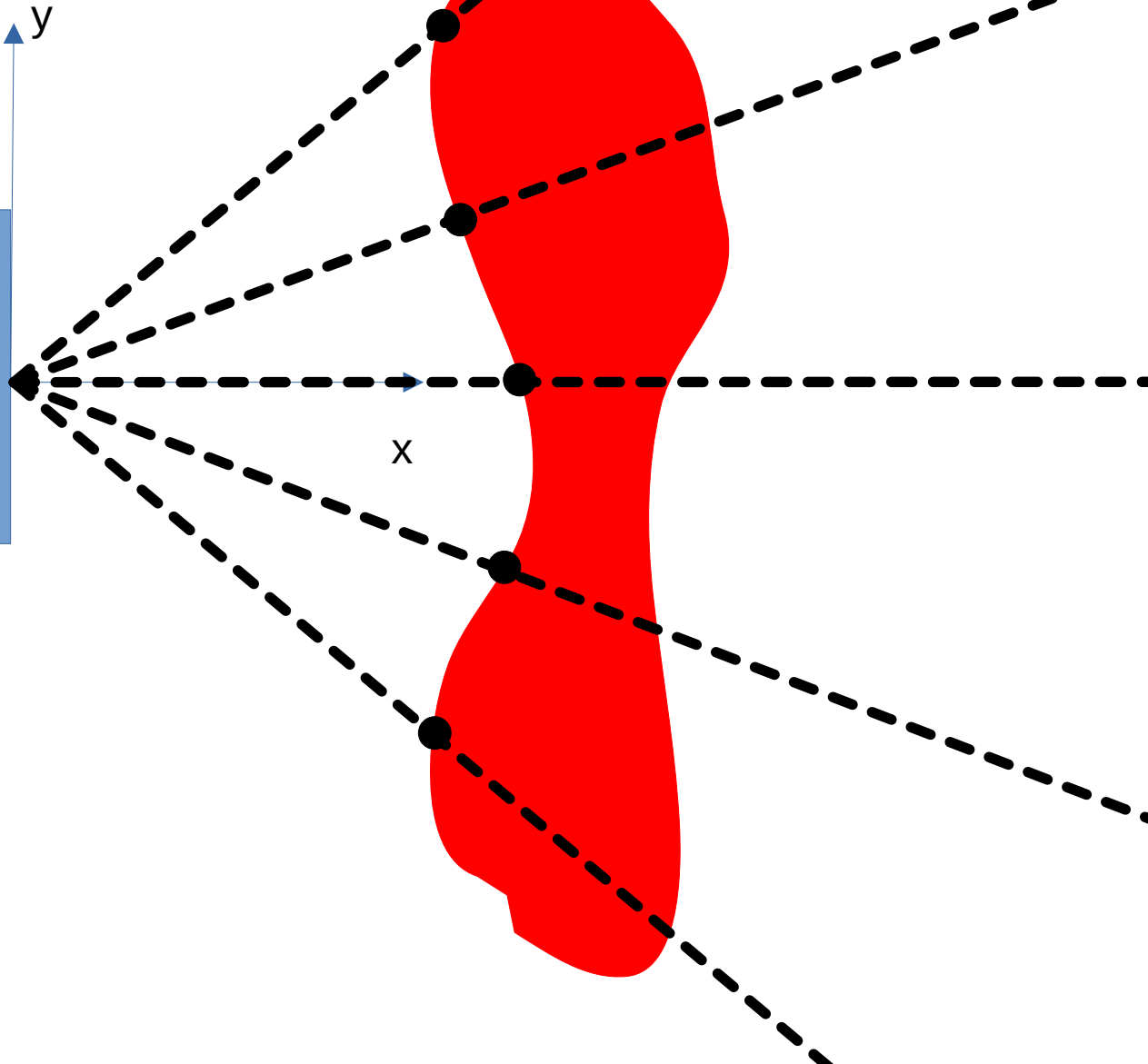


Nice to have's:

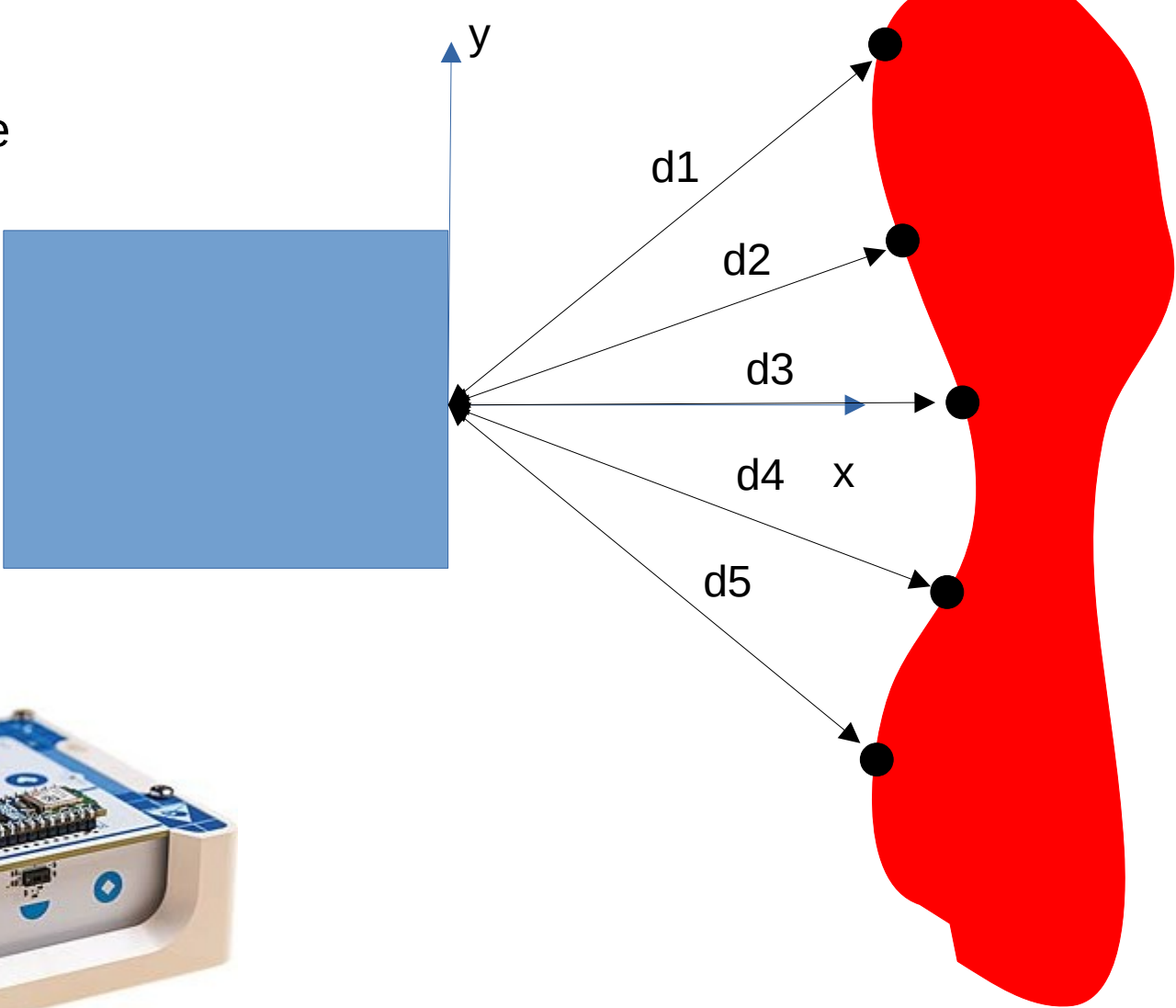
- Expressive enough to try crazy research ideas
- Not rigidly tied to any particular ecosystem
- Not too much performance penalty
- Free to use
- Auditable
- **Alive/Stable across multiple generations of PhDs**

- Active since 2010
- 2 main developers
 - Joris Gillis
 - Joel Andersson
- Commercial support

Alvik exercise



Alvik exercise

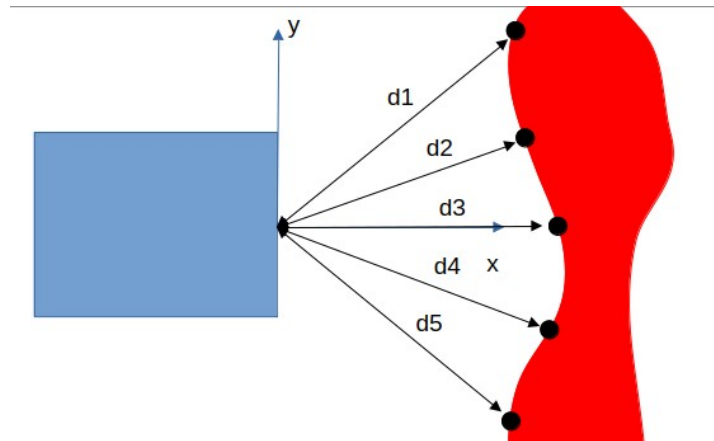


B

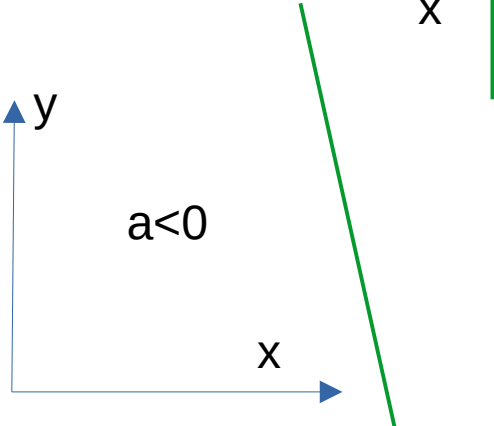
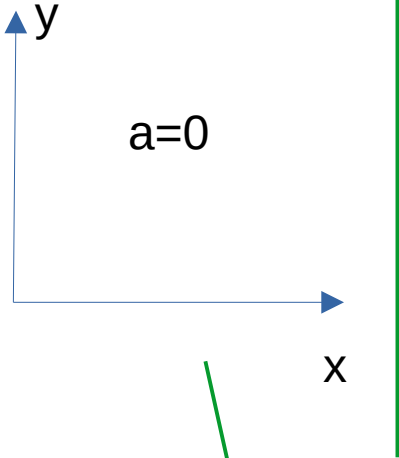
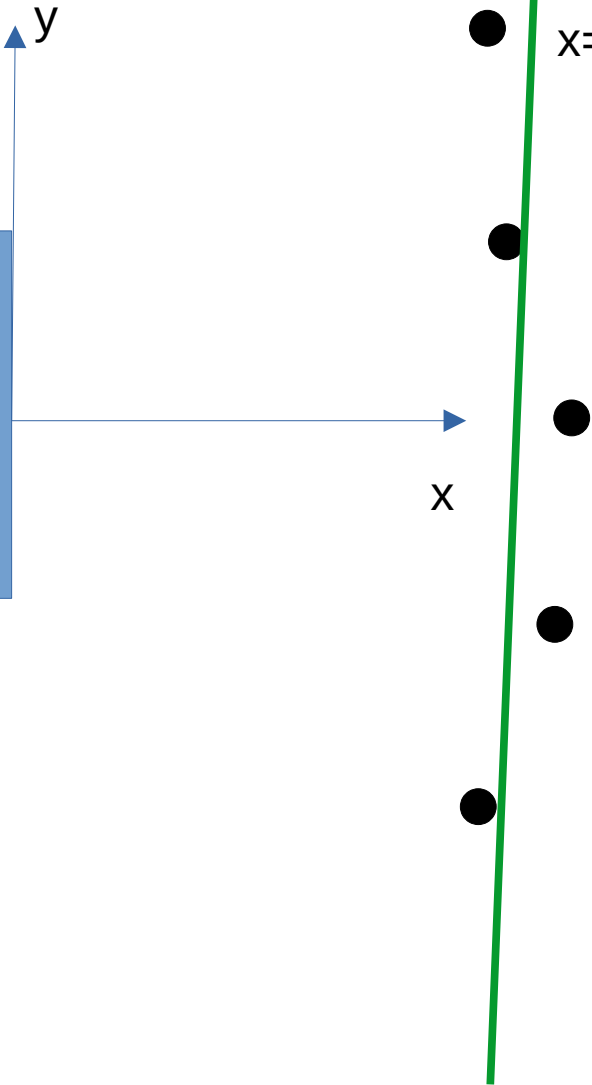
Alvik exercise

```
angles = ca.vertcat(-45,-22,0,22,45)  
angled_rad = angles/180*np.pi
```

```
d = ca.vertcat(6.7,6.2,6,5.9,5.7)
```



Alvik exercise



Alvik exercise

```
angles = ca.vertcat(-45,-22,0,22,45)
angled_rad = angles/180*np.pi
```

```
d = ca.vertcat(6.7,6.2,6,5.9,5.7)
```

```
opti = ca.Opti()
```

```
opti.minimize(...)
```

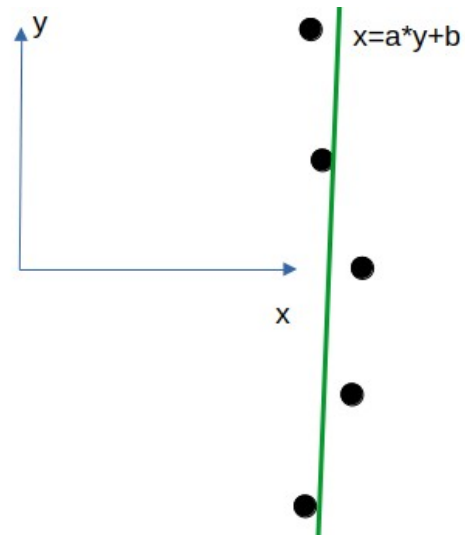
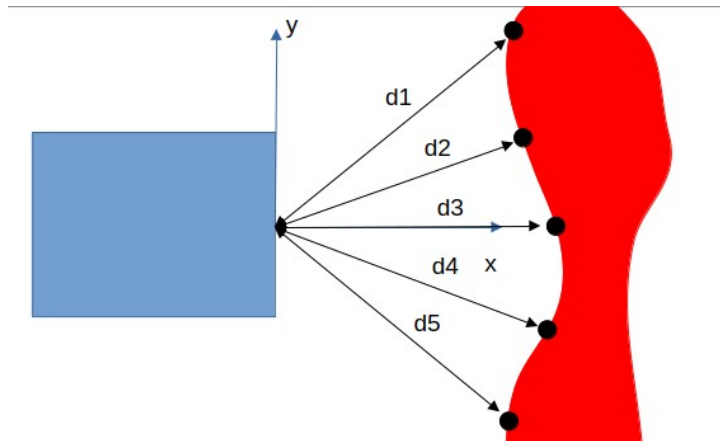
```
opti.solver("ipopt")
```

```
sol = opti.solve()
```

```
print(sol.value(a))
```

Expected answer

-0.066



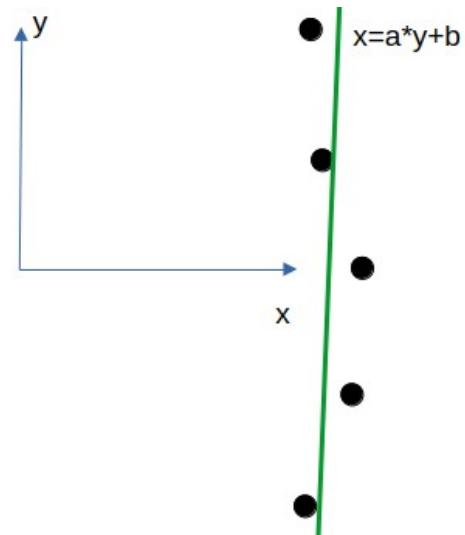
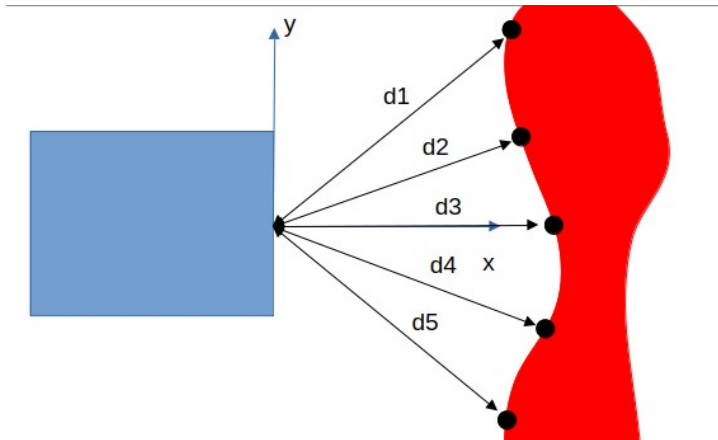
Alvik exercise

```
d = opti.parameter(5)
```

```
...
```

```
print(fitter) # fitter:(d[5])-(a,b) MXFunction
```

```
fitter.generate('fitter.c', {"with_header":True})
```



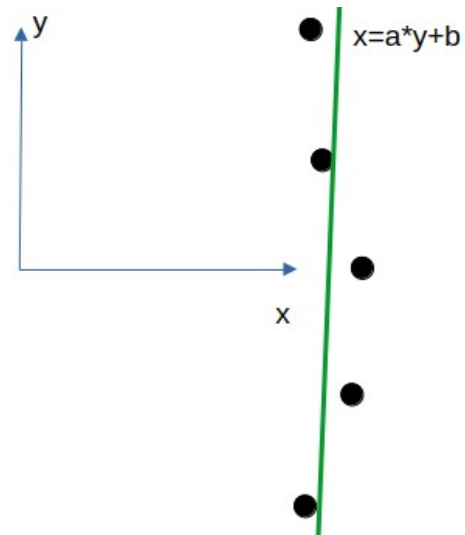
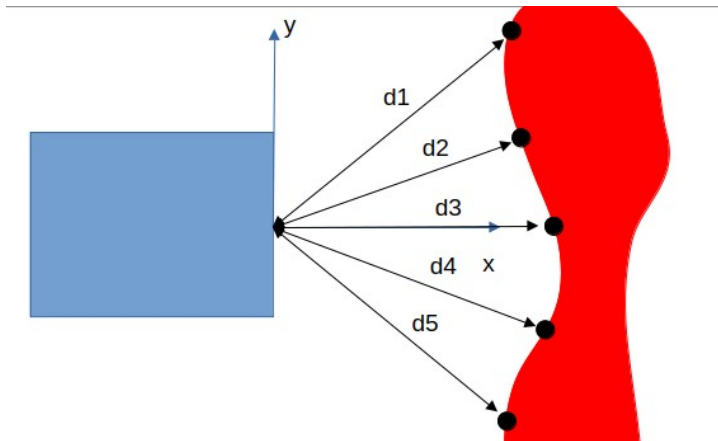
Alvik exercise

```
d = opti.parameter(5)
```

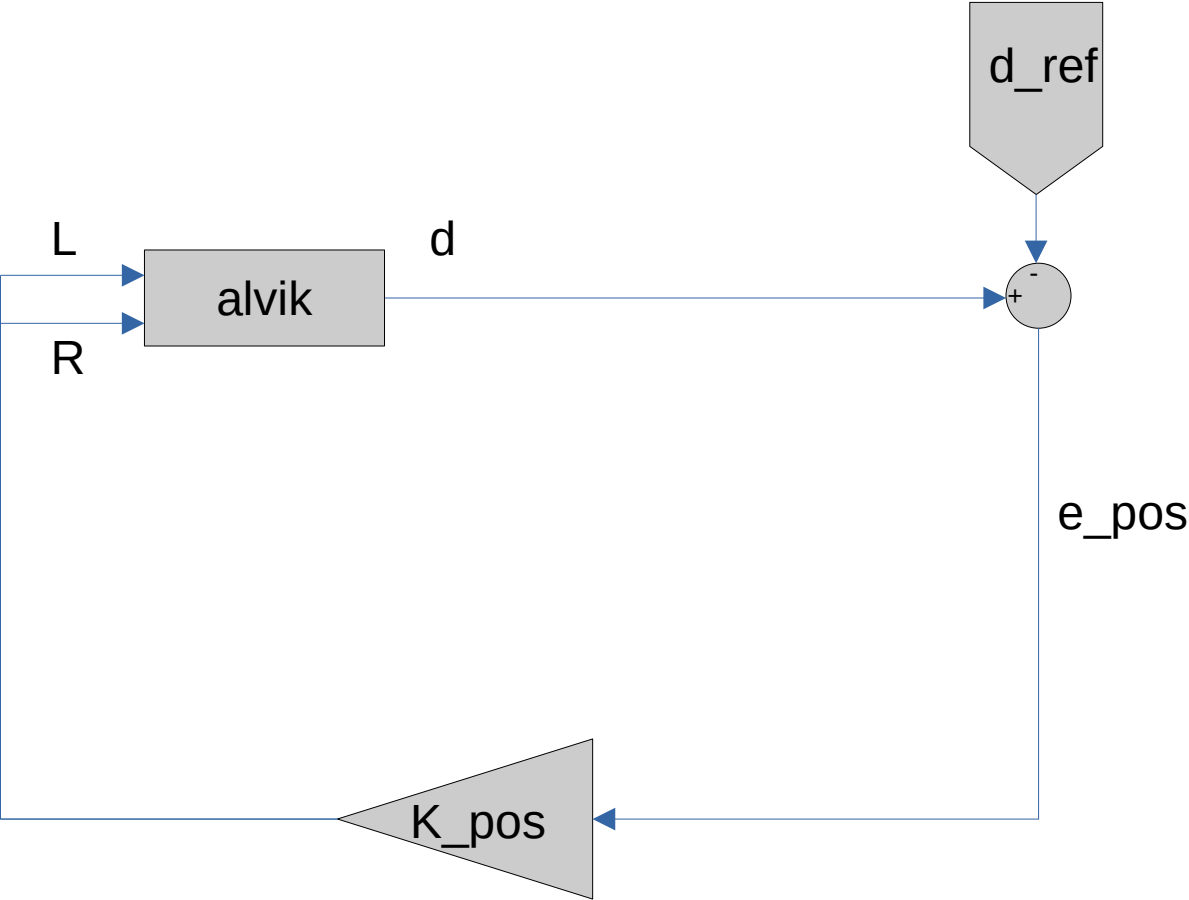
```
...
```

```
print(fitter) # fitter:(d[5])-(a,b) MXFunction
```

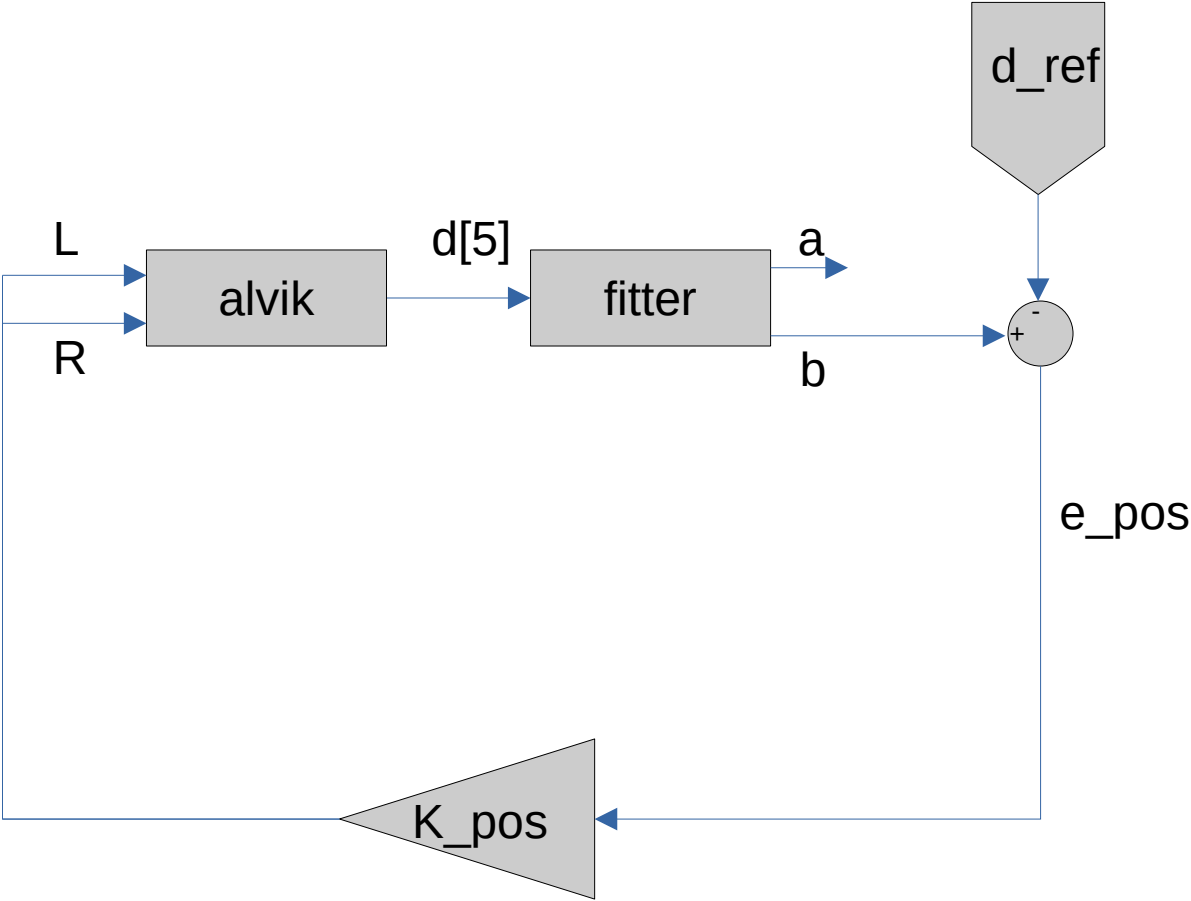
```
fitter.generate('fitter.c', {"with_header": True})
```



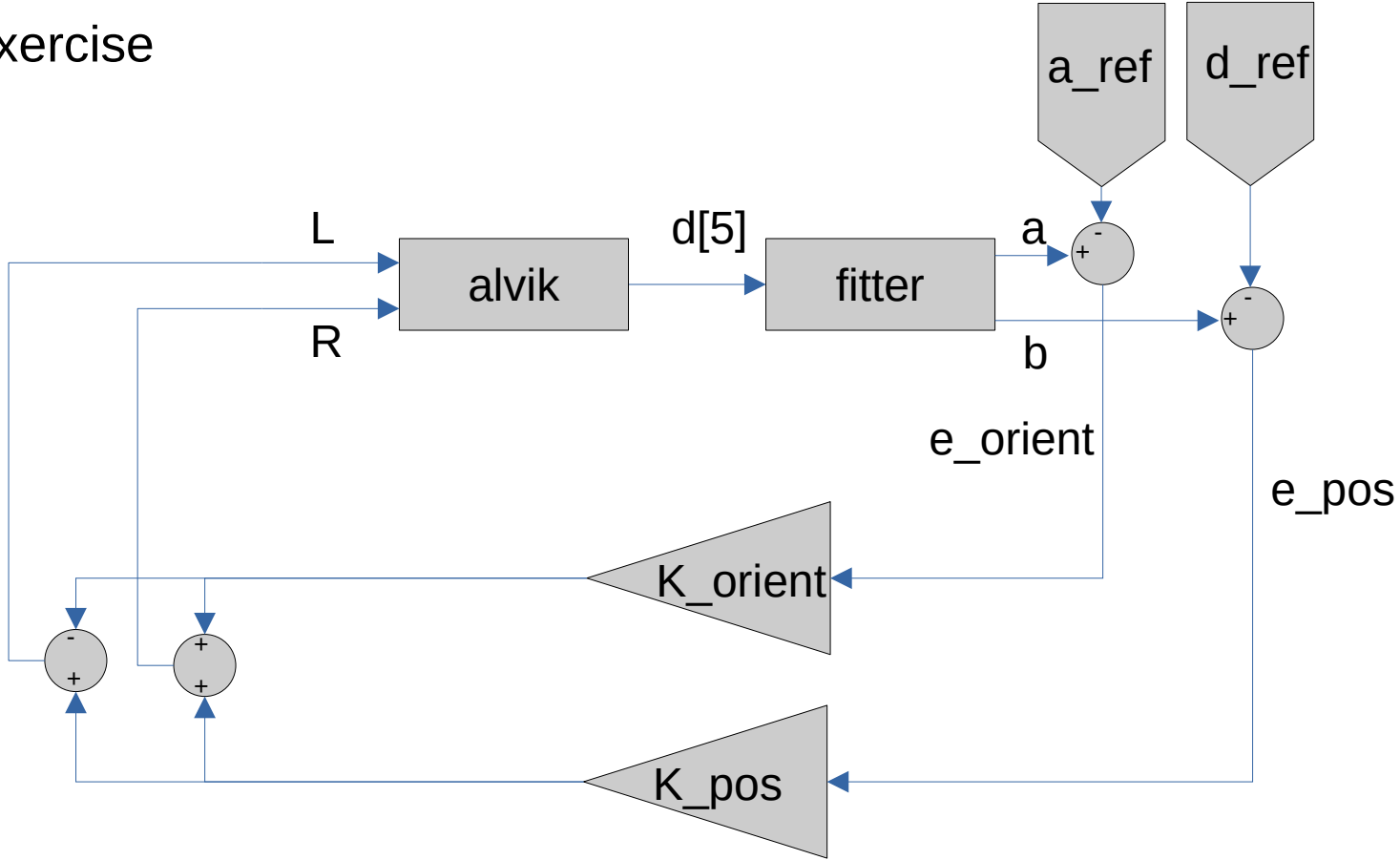
Alvik exercise



Alvik exercise



Alvik exercise



Algorithmic differentiation

- CasADi uses source-code-transformation AD
 - Source code = expression graph
 - Infinite order
- Both forward and reverse mode AD

Power of reverse mode AD

```
import casadi as ca

K = 1000

x = ca.SX.sym("x",K)

e = x[0]
for i in range(K):
    e = e*ca.sin(e)

y = e # 1-by-1

print(ca.jacobian(y,x).shape)
      1-by-1000
```

Power of reverse mode AD

```
import casadi as ca
```

```
N = 1000
```

```
x = ca.SX.sym("x",K)
```

```
e = x[0]
```

```
for i in range(K):  
    e = ca.sin(e)*x[i]
```

```
y = e # 1-by-1
```

```
print(ca.n_nodes(y))
```

3000

```
print(ca.n_nodes(ca.jacobian(y,x)))
```

6998

- Original cost: C
- Reverse mode code: $(2..4)*C$

Powers the Deep Learning revolution

In constrained optimization, forward mode is important too

```
N = 1000
```

```
x = ca.SX.sym("x")  
r = ca.SX.sym("r", K)
```

```
g = [x]  
for i in range(K):  
    g.append(ca.sin(r[i]*g[-1]))  
y = ca.vcat(g)
```

```
y = e # N-by-1
```

```
print(ca.n_nodes(y))
```

```
print(ca.n_nodes(ca.jacobian(y,x)))
```

- Original cost: C
- Forward mode code: (2..3)*C

3001

6000

Two extreme cases

```
ca.jacobian(y,x,  
{"helper_options" : {"enable_reverse":False}})
```

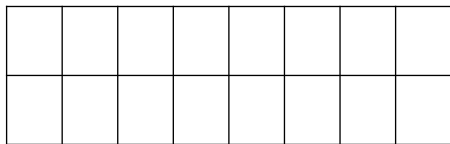
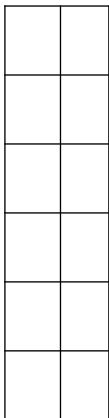
		Case A	Case B
N	Size of y	1	1000
M	Size of x	1000	1
C	Cost of y(x)	3000	3001
	Cost of jacobian(y,x) N-by-M	6998	6000
	Cost of jacobian(y,x) via reverse mode	6998	1011666
	Cost of jacobian(y,x) via forward mode	1010306	6000

Two extreme cases

		Case A	Case B
N	Size of y	1	1000
M	Size of x	1000	1
C	Cost of $y(x)$	$O(N)$	$O(M)$
	Cost of jacobian(y,x) N-by-M	$O(N)$	$O(M)$
	Cost of jacobian(y,x) via reverse mode	$O(N)$	$O(M^2)$
	Cost of jacobian(y,x) via forward mode	$O(N^2)$	$O(M)$

General case

Jacobian N-by-M



$N \gg M \rightarrow$ forward mode $O(M \cdot C)$

$N \ll M \rightarrow$ reverse mode $O(N \cdot C)$

But: sparsity can be exploited

- Effective number of reverse or forward sweeps is determined by sparsity
 - “Jacobian coloring”

$N \gg M \rightarrow$ forward mode $(2..3) \cdot (1..M) \cdot C$

$N \ll M \rightarrow$ reverse mode $(2..4) \cdot (1..N) \cdot C$

CasADi uses a simple heuristic

Online CasADi training

- Cohort October 2026 will open soon (at the CasADi 3.8 release)
- Keep an eye on the CasADi website (newsletter)

CasADi session outline

- Software stacks
- CasADi
- **Rockit**

```
U = opti.variable(4)
```

```
x0 = 2  
x1 = F(x0,U[0])  
x2 = F(x1,U[1])  
x3 = F(x2,U[2])  
x4 = F(x3,U[3])
```

```
opti.subject_to(x1>=0)  
opti.subject_to(x2>=0)  
opti.subject_to(x3>=0)  
opti.subject_to(x4==3)
```

```
X = ca.vertcat(x0,x1,x2,x3,x4)
```

```
opti.minimize(ca.sumsqr(U)  
+ca.sumsqr(X))
```

```
opti.solver('ipopt')  
sol = opti.solve()
```

```
U = opti.variable(N)
```

```
x = 2  
X = [x]
```

```
for k in range(N):  
    x = F(x,U[k])  
    X.append(x)
```

```
X = ca.hcat(X)
```

```
opti.subject_to(X[1:-1]>=0)  
opti.subject_to(X[-1]==3)
```

```
opti.minimize(ca.sumsqr(U)  
+ca.sumsqr(X))
```

```
opti.solver('ipopt')  
sol = opti.solve()
```

Optimal control with CasADi Opti

Single-shooting

```
U = opti.variable(N)
```

```
x = 2  
X = [x]
```

```
for k in range(N):  
    x = F(x,U[k])  
    X.append(x)
```

```
X = ca.hcat(X)
```

```
opti.subject_to(X[1:-1]>=0)  
opti.subject_to(X[-1]==3)
```

Multiple-shooting

```
X = opti.variable(N+1)  
U = opti.variable(N)
```

```
for k in range(N):  
    opti.subject_to(F(X[k],U[k])==X[k+1])
```

```
opti.subject_to(X[0]==2)  
opti.subject_to(X[1:-1]>=0)  
opti.subject_to(X[-1]==3)
```